

1.5. Massiv shakli va uning o'lchamlarini o'zgartirish (Shaping & Reshaping in Array)

Massivning **shakli (shape)** har bir o'lchamdagi elementlar soni sifatida ta'riflanadi. Unga massiv o'lchamlarini ifodalovchi kortejni (tuple) qaytaruvchi `shape` atributi orqali kirish mumkin. Ushbu paragrafda NumPy massivining shaklini qanday o'zgartirishni o'rganamiz. Bu jarayon massiv shaklini qayta qurish (**reshaping**), yassilash (**flattening**) va muayyan vazifalarga moslashtirish uchun massiv tuzilmasini modifikatsiya qilishni o'z ichiga oladi.

1.5.1. Massiv shaklini boshqarish (Shape Manipulation in NumPy)

Massivning **shakli (shape)** har bir o'lchamdagi elementlar soni sifatida ta'riflanishi mumkin. **O'lcham (dimension)** — bu massivning alohida elementini ko'rsatish (aniqlash) uchun talab qilinadigan indekslar yoki pastki belgilar sonidir.

Massiv shaklini qanday olish mumkin?

NumPy-da biz `shape` deb nomlangan atributdan foydalanamiz, u **kortej (tuple)** qaytaradi; kortej elementlari massivning mos keladigan o'lchamlari uzunligini bildiradi.

- **Sintaksis:** `numpy.shape(massiv_nomi)`
- **Parametrlar:** Massiv parametr sifatida uzatiladi.
- **Qaytariladigan qiymat:** Elementlari massiv o'lchamlarining uzunligini ko'rsatuvchi kortej.

NumPy-da shakl manipulyatsiyasi (Shape Manipulation)

Quyida Python-dagi NumPy kutubxonasida shakllar bilan ishlashni tushunish uchun misollar keltirilgan:

1-misol: Massivlar shakli Ko'p o'lchamli massivning shaklini chop etish. Ushbu misolda mos ravishda 2D va 3D massivlarni ifodalovchi `arr1` va `arr2` nomli ikkita NumPy massivi yaratilgan. Har bir massivning shakli (shape) chop etilib, ularning o'lchamlari va har bir o'lcham bo'yicha hajmi ko'rsatilgan.

```
import numpy as np
```

```
# 2 o'lchamli (2-D) massiv yaratish
arr1 = np.array([[1, 3, 5, 7], [2, 4, 6, 8]])

# 3 o'lchamli (3-D) massiv yaratish
arr2 = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

print(arr1.shape)
print(arr2.shape)

(2, 4)
(2, 2, 2)
```

2-misol: ndmin yordamida massiv shaklini belgilash

Ushbu misolda biz 2, 4, 6, 8, 10 qiymatlariga ega vektordan foydalangan holda `ndmin` parametri orqali massiv yaratamiz va oxirgi o'lcham qiymatini tekshiramiz.

```
# 6 o'lchamli massiv yaratish
# ndmin parametridan foydalangan holda
arr = np.array([2, 4, 6, 8, 10], ndmin=6)

# Massivni chop etish
print(arr)

# Oxirgi o'lcham qiymati 5 ekanligini
# va jami 6 ta o'lchamni tekshirish
print('massiv shakli :', arr.shape)

[[[[[[[ 2  4  6  8 10]]]]]]]]
massiv shakli : (1, 1, 1, 1, 1, 5)
```

Izoh: `ndmin=6` parametri NumPy-ga massiv kamida 6 o'lchamli bo'lishi kerakligini aytadi. Dastlabki 5 ta o'lcham "to'ldiruvchi" (har birida 1 tadan element) bo'lib xizmat qiladi, oxirgi (oltinchi) o'lcham esa biz kiritgan 5 ta elementni o'z ichiga oladi. Shuning uchun natija `(1, 1, 1, 1, 1, 5)` ko'rinishida chiqadi.

3-misol: Kortejlar (tuples) massivining shakli

Ushbu misolda biz har bir elementi kortejdan iborat bo'lgan NumPy massivini yaratamiz va bunday massivning shakli (shape) qanday aniqlanishini ko'rsatib beramiz.

```
# Kortejlar ro'yxatidan massiv yaratish
array_of_tuples = np.array([(1, 2), (3, 4), (5, 6), (7, 8)])

# Massivni ko'rsatish
print("Kortejlar massivi:")
```

```
print(array_of_tuples)

# Shaklni aniqlash va ko'rsatish
shape = array_of_tuples.shape
print("\nMassiv shakli:", shape)
```

Kortejlar massivi:

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

Massiv shakli: (4, 2)

Izoh: NumPy kortejlar ro'yxatini qabul qilganda, ularni avtomatik ravishda 2 o'lchamli massivga (matritsaga) aylantiradi. Har bir kortej massivning bitta qatori deb hisoblanadi. Bu yerda 4 ta kortej bor va har birida 2 tadan element mavjud, shuning uchun massiv shakli **(4, 2)** bo'ladi.

1.5.2. Massiv shaklini o'zgartirish (Array Reshaping)

NumPy-da shaklni o'zgartirish (**Reshaping**) — bu mavjud massivning ma'lumotlarini o'zgartirmasdan, uning o'lchamlarini modifikatsiya qilishdir. Buning uchun `reshape()` funksiyasidan foydalaniladi. U elementlarni yangi shaklga qayta tartiblaydi, bu esa mashinali o'rganish, matritsa amallari va ma'lumotlarni tayyorlashda juda foydalidir.

1-misol: Ushbu misol jami elementlar soniga mos keladigan qator va ustunlarni ko'rsatish orqali 1-D massivni 2-D massivga aylantiradi.

```
a = np.array([1, 2, 3, 4, 5, 6])
r = a.reshape(2, 3)
print(r)
```

```
[[1 2 3]
 [4 5 6]]
```

Izoh: `a.reshape(2, 3)` 6 ta elementni 2 ta qator va 3 ta ustunga joylashtirib, 2-D matritsani hosil qiladi.

Sintaksis `array.reshape(shape)`

- **Parametr:** `shape` - Yangi o'lchamlarni belgilaydigan kortej (tuple). Bir o'lcham **-1** bo'lishi mumkin, bunda NumPy jami elementlar sonidan kelib

chiqib, ushbu o'lchamni avtomatik hisoblab oladi.

- **Qaytaradi:** Yangi shaklga ega bo'lgan `ndarray`.

2-misol: Ushbu misol asl elementlarni har biri teng o'lchamdagi 2-D qismlardan iborat bloklarga guruhlash orqali 3-D massiv yaratadi.

```
a = np.array([1, 2, 3, 4, 5, 6, 7, 8])
r = a.reshape(2, 2, 2)
print(r)
```

```
[[[1 2]
   [3 4]]

  [[5 6]
   [7 8]]]
```

Izoh: `a.reshape(2, 2, 2)` massivni har biri 2x2 o'lchamli matritsadan iborat bo'lgan 2 ta blokka o'zgartiradi va 3-D tuzilmani hosil qiladi.

3-misol: Ushbu misol bitta o'lcham noma'lum bo'lganda **-1** dan foydalanishni ko'rsatadi. NumPy yetishmayotgan o'lchamni avtomatik ravishda hisoblab chiqadi.

```
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
r = a.reshape(3, -1)
print(r)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

Izoh: `a.reshape(3, -1)` NumPy-ga 3 ta qator yaratishni buyuradi va u qolgan o'lchamni 4 ta ustun deb hisoblaydi, chunki $12 \div 3 = 4$.

1.5.3. NumPy massivlarini tekislash va tartibni o'zgartirish (Flattening and Changing Order)

`flatten()` funksiyasi ko'p o'lchamli **NumPy massivini** bir o'lchamli massivga o'tkazish uchun ishlatiladi. U ma'lumotlarning yangi nusxasini yaratadi, shuning uchun asl massiv o'zgarishsiz qoladi. Agar massivingizda qatorlar va ustunlar yoki undan ham ko'proq o'lchamlar bo'lsa, `flatten()` barcha qiymatlarni birin-ketin to'g'ri chiziqli ro'yxatga joylashtiradi.

Misol:

```
a = np.array([[5, 6], [7, 8]])
res = a.flatten()
print(res)
```

```
[5 6 7 8]
```

Izoh: Ushbu kod 2D NumPy massivini yaratadi va `flatten()` yordamida uni qatorlar tartibida (row-major order) 1D massivga aylantiradi. U asl massivni o'zgartirmasdan, yangi `[5, 6, 7, 8]` massivini qaytaradi.

`ndarray.flatten()` sintaksisi

```
array.flatten(order='C')
```

Parametrlar: `order` ({'C', 'F', 'A', 'K'}) - ixtiyoriy

- **'C':** Qatorlar bo'yicha (C-uslubi) tartiblash.
- **'F':** Ustunlar bo'yicha (Fortran-uslubi) tartiblash.
- **'A':** Agar massiv xotirada Fortran tartibida bo'lsa ustunlar bo'yicha, aks holda qatorlar bo'yicha.
- **'K':** Elementlar xotirada qanday ketma-ketlikda joylashgan bo'lsa, shu tartibda yassilash.

Qaytaradi: Ko'rsatilgan tartib bo'yicha yassilangan kiruvchi massivning 1D nusxasini.

`ndarray.flatten()` uchun misollar

1-misol: Ushbu misolda biz 2D NumPy massivini ustunlar bo'yicha (column-major order, shuningdek, Fortran-uslubidagi tartib deb ham ataladi) yassilash uchun `flatten()` funksiyasini `'F'` parametri bilan ishlatamiz. Bu funksiya elementlarni qatorlar bo'yicha emas, balki ustunma-ustun o'qiydi deganidir.

```
a = np.array([[5, 6], [7, 8]])
res = a.flatten('F')
print(res)
```

```
[5 7 6 8]
```

Izoh: `flatten('F')` massivni ustunlar bo'yicha (Fortran-uslubida) 1D massivga aylantiradi. Natija `[5, 7, 6, 8]` bo'lib, elementlar ustunma-ustun tartibda joylashgan.

2-misol: Ushbu misolda biz ikkita 2D NumPy massivini yassilashni va so'ngra ularni NumPy-ning `concatenate()` funksiyasi yordamida bitta 1D massivga birlashtirishni ko'rib chiqamiz.

```
a = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([[7, 8, 9], [10, 11, 12]])

res = np.concatenate((a.flatten(), b.flatten()))
print(res)

[ 1  2  3  4  5  6  7  8  9 10 11 12]
```

Izoh: Ushbu kod `a` va `b` nomli ikkita 2D NumPy massivini 1D massivga yassilaydi va so'ngra ularni birlashtiradi. Natijada har ikkala massivning yassilangan elementlarini o'z ichiga olgan yagona 1D massiv hosil bo'ladi.

3-misol: Ushbu misolda biz 2D NumPy massivini yassilashni va so'ngra uning shakliga mos keladigan, nollar bilan to'ldirilgan yangi 1D massivni yaratishni ko'rib chiqamiz.

```
a = np.array([[1, 2, 3], [4, 5, 6]])
res = np.zeros_like(a.flatten())
print(res)

[0 0 0 0 0 0]
```

Izoh: Ushbu kod `np.zeros_like()` funksiyasidan foydalanib, bir xil shakldagi, nollar bilan to'ldirilgan yangi massiv yaratadi. Natija `a` massivining yassilangan shakliga mos keladigan nollardan iborat 1D massivdir.

4-misol: Ushbu misolda biz 2D NumPy massivini yaratamiz. So'ngra yassilangan massivdagi barcha elementlar orasidan eng katta qiymatni topish uchun `max()` metodini qo'llaymiz.

```
a = np.array([[4, 12, 8], [5, 9, 10], [7, 6, 11]])
res = a.flatten().max()
print(res)

12
```

Izoh: Ushbu kod `a` 2D NumPy massivini 1D massivga yassilaydi va keyin `.max()` yordamida eng katta qiymatni topadi. Natija **12** bo'lib, u yassilangan massivdagi eng katta qiymatdir.

1-daraja: Asosiy tushunchalar

1. NumPy massivining shakli (`shape`) deganda nima tushuniladi va unga qaysi atribut orqali murojaat qilinadi?
2. `shape` atributi qaytaradigan kortej (tuple) elementlari nimani ifodalaydi?
3. Massiv yaratishda `ndmin` parametridan foydalanishning asosiy maqsadi nimadan iborat?
4. NumPy kortejlar ro'yxatidan massiv yaratganda, ularni avtomatik ravishda qanday shaklga o'tkazadi?

2-daraja: Funksiyalarning ishlash mexanizmi

5. `reshape()` funksiyasi massiv shaklini o'zgartirganda undagi ma'lumotlarga qanday ta'sir ko'rsatadi?
6. `reshape()` metodida o'lchamlardan biri sifatida **-1** qiymati berilsa, NumPy bu o'lchamni qanday aniqlaydi?
7. `flatten()` funksiyasi vazifasini tushuntiring va u asl massivni o'zgartiradimi yoki yo'qmi?
8. `flatten()` metodidagi `order='C'` va `order='F'` parametrlari o'rtasidagi asosiy farq nimada?

3-daraja: Amaliy tahlil va shakl manipulyatsiyasi

9. 1-D massivni 2-D yoki 3-D shaklga o'tkazishda elementlar soni va yangi shakl o'rtasidagi mantiqiy bog'liqlikni izohlang.
10. `np.zeros_like()` funksiyasini yassilangan massiv bilan birga qo'llash qanday natija beradi?
11. Nima uchun mashinali o'rganish va matritsa amallarida massiv shaklini qayta qurish (`reshaping`) muhim hisoblanadi?
12. Massivni yassilash jarayonida `order='K'` parametri elementlarni xotirada joylashish tartibiga ko'ra qanday tartiblaydi?